

A SUMMER LEARNING PLAYBOOK

The Python Playbook

From your very first line of code to designing real object-oriented programs — and building an AI-powered app of your own — in four focused weeks.

```
>>> message = "Hello, future programmer."  
>>> print(message)  
Hello, future programmer.
```

Learn · Practice · Build

Self-guided · No prior experience required

4 Weeks

~ 2 hours a day

START HERE

How to Use This Playbook

This is not a book you read once. It is a workshop you live in for four weeks. Every page is built to be *done*, not just understood — because nobody ever learned to swim by reading about water.

You hold a complete, self-contained course. You do not need to buy anything, sign up for anything, or have any background in computing. You need a computer, an internet connection for the final week, and the single most important ingredient: a habit of showing up every day.

■ The rhythm of a day

Each week is split into bite-sized topics. A healthy daily session of around two hours looks like this:

Phase	Time	What you do
Read & Type	40 min	Read a topic. Type every code example yourself — never copy-paste. Your fingers need to learn the shapes.
Check	15 min	Answer the Knowledge Check questions out loud or on paper before peeking at the answer key.
Practice	50 min	Do the Exercises . Struggle a little. Struggling is the sound of your brain growing.
Reflect	15 min	Write one sentence in a notebook: " <i>Today I learned that...</i> ". This locks it in.

■ The signposts you will see

Throughout the playbook, coloured boxes guide you. Learn to recognise them:

ANALOGY

A bridge from something you already understand in the real world to a new programming idea. Analogies are scaffolding — useful while building, but remember the real thing is the code.

WHERE THE WORD COMES FROM

The story behind a keyword or concept. Knowing that **string** literally means a *string of beads* makes it stick — and reveals that programmers borrowed almost every term from older crafts.

PRO TIP

A small habit or shortcut that working programmers use. Adopt these early.

WATCH OUT

A classic mistake that trips up nearly every beginner. Forewarned is forearmed.

✓ KNOWLEDGE CHECK

Quick questions to test whether an idea actually landed. Answer before reading on. Full answers are in **Appendix A**.

 EXERCISE

Hands-on tasks. Each is tagged **WARM-UP** **CORE** or **STRETCH** by difficulty. Solutions and hints are in **Appendix B**.

THE ONE RULE

Type the code. Run the code. Break the code on purpose. Change a number, delete a line, see what error appears. A program you have broken and fixed is a program you understand. You cannot harm your computer by running Python — the worst that happens is an error message, and error messages are just the language talking back to you.

THE MAP

Your Four-Week Roadmap

Everything here builds toward one destination: thinking in **objects**. We climb there one solid step at a time. You will never meet an idea that depends on something you have not already mastered.

Week	Theme	You will be able to...
Week 1 Foundations	Speaking to the Machine	Store data in variables, work with text and numbers, and make your program take different paths with decisions.
Week 2 Structure	Building Reusable Machinery	Repeat work with loops, organise data in lists and dictionaries, and package logic into reusable functions.
Week 3 Modelling	Object-Oriented Programming	Design your own data types with classes, and use the four pillars — encapsulation, inheritance, polymorphism, abstraction.
Week 4 The Real World	Files, Errors & an AI Capstone	Handle files and errors gracefully, talk to the internet, and build a real OpenAI-powered application using everything you have learned.

ANALOGY — BUILDING A HOUSE

Week 1 is learning to **lay bricks** (single instructions). Week 2 is learning to build **walls and rooms** (loops and functions that organise many bricks). Week 3 is becoming the **architect** who designs whole houses from reusable blueprints (classes). Week 4 is **connecting the house to the city** — water, electricity, and the wider world. You cannot wire a house before its walls exist, which is exactly why the order matters.

Contents

Front Matter

How to Use This Playbook	3
Your Four-Week Roadmap	5
Setting Up Your Workshop	7

Week 1 — Foundations: Speaking to the Machine

Programs, your first line, variables, data types	11
Operators, strings, decisions, project	16

Week 2 — Structure: Building Reusable Machinery

Loops, lists, tuples, dictionaries, sets	24
Functions, scope, comprehensions, project	31

Week 3 — Modelling: Object-Oriented Programming

Classes, objects, the four pillars, dunder methods	39
--	----

Week 4 — The Real World & AI Capstone

Modules, files, errors, the OpenAI API, capstone	52
--	----

Appendices

A · Knowledge Check Answers	64
B · Exercise Solutions & Hints	66
C · Quick-Reference Cheat Sheet	69
D · Glossary of Word Origins	71

SETUP

Setting Up Your Workshop

A carpenter needs a bench before the first cut. You need three things: Python itself, a place to write code, and a way to run it. Twenty minutes here saves twenty hours later.

■ Step 1 — Install Python

Python is free and open-source. Go to python.org/downloads and download the latest version (anything 3.11 or newer is perfect). During installation on Windows, there is one box you **must** tick:

WATCH OUT — THE ONE BOX EVERYONE MISSES

On Windows, tick “Add Python to PATH” on the very first installer screen before clicking Install. PATH is the list of places your computer searches for programs. If Python is not on it, your computer will claim Python does not exist even though you just installed it. On macOS and Linux this is handled automatically.

To confirm it worked, open a **terminal** (Command Prompt or PowerShell on Windows, Terminal on macOS/Linux) and type:

```
python --version
Python 3.12.4
```

If you see a version number, you are ready. (On some systems the command is `python3` instead of `python` — try that if the first fails.)

WHERE THE WORD COMES FROM — “PYTHON”

The language is **not** named after the snake. Its creator, Guido van Rossum, was a fan of the British comedy troupe Monty Python's *Flying Circus* and wanted a name that was short, unique, and a little mysterious. The snake imagery came later, from publishers. This is why Python culture is famously playful — official documentation is sprinkled with references to spam, eggs, and silly sketches.

■ Step 2 — Get an editor

You can write Python in any text editor, but a proper **code editor** highlights your code in colour, points out mistakes, and runs programs at the press of a button. The standard choice for beginners and professionals alike is **Visual Studio Code** (free, from code.visualstudio.com). After installing it, open its Extensions panel and install the official **Python** extension by Microsoft.

ANALOGY — EDITOR VS. INTERPRETER

Think of the **editor** as your *writing desk* — where you compose your instructions. The **Python interpreter** is the *translator* who reads what you wrote and carries out each instruction, one line at a time, turning your English-like text into actions the machine understands. You write at the desk; the translator does the work.

■ Step 3 — The two ways to run code

You will use both throughout this playbook:

Way	What it is & when to use it
The REPL (interactive shell)	Type <code>python</code> in your terminal and you get a <code>>>></code> prompt. Type one line, press Enter, see the result instantly. Perfect for quick experiments — like a calculator that speaks Python. REPL stands for Read-Eval-Print Loop : it reads your line, evaluates it, prints the answer, and loops back for more.
A script file (<code>.py</code> file)	Save your code in a file like <code>hello.py</code> , then run all of it at once. This is how real programs are built and saved. Run it from the terminal with <code>python hello.py</code> or press the ▶ Run button in VS Code.

■ Your first program

Create a file called `hello.py`, type this single line (don't copy-paste!), and run it:

```
print("Hello, future programmer.")
```

If you see the words appear, congratulations — you have written and run a real program. Everything from here is just learning what else you can put inside, before, and after lines like this one.

PRO TIP — THE PLAYGROUND OPTION

If installing software feels daunting right now, you can start *today* with zero setup at an online Python playground in your browser (search “online Python interpreter”). It is perfect for Week 1. But do install Python properly by Week 4 — the AI capstone needs to run on your own machine.

✓ KNOWLEDGE CHECK — SETUP

1. What does adding Python “to PATH” actually let your computer do?
2. What is the difference between the REPL and a `.py` script file? When would you reach for each?
3. What do the four letters in **REPL** stand for?

01

WEEK ONE

Foundations: Speaking to the Machine

Before you can build anything, you must learn the alphabet of instructions: how to store information, how to handle words and numbers, and how to make a program choose between paths. This is the week you stop being someone who uses software and become someone who writes it.

- ▶ Understand what a program really is and how Python runs it
- ▶ Store and label information using variables
- ▶ Work confidently with the core data types: text, whole numbers, decimals, and true/false
- ▶ Combine values using operators
- ▶ Slice, search, and shape text (strings)
- ▶ Make your program decide between different paths with `if` / `elif` / `else`
- ▶ Build your first interactive program from scratch

1.1 What Is a Program, Really?

A program is a list of instructions, written down in advance, that a machine follows exactly — no more, no less, in precisely the order given.

ANALOGY — A RECIPE FOR A VERY LITERAL COOK

Imagine writing a recipe for a cook who has **never cooked before, takes every word literally, and never guesses**. If you write “add salt” without saying how much, they freeze. If you say “beat the eggs” they may fetch a stick. Computers are exactly this cook: dazzlingly fast, perfectly obedient, and utterly without common sense. **Programming is the art of being unambiguous**. Most beginner bugs are simply moments where you assumed the cook would “just know” something.

Python runs your instructions **top to bottom, one line at a time**, like reading a list. This single idea — *sequential execution* — is the floor everything else stands on. Later we will learn to make the cook repeat steps (loops), skip steps (decisions), and follow named sub-recipes (functions), but underneath it is always: read a line, do it, move to the next.

■ The print function: making the machine speak

You have already met `print()`. It is how your program talks back to you, putting text on the screen. The words inside the quotation marks are shown exactly:

```
print("Learning Python this summer.")
print("One instruction per line.")
```

```
Learning Python this summer.
One instruction per line.
```

WHERE THE WORD COMES FROM — "PRINT"

Long before screens, a computer's only output was a **physical printer** hammering characters onto paper. The command to send text out was naturally called `print`. The paper is gone, but the verb survived — a tiny fossil of computing history living in code you write today. Programming vocabulary is full of these fossils.

■ Comments: notes the machine ignores

Anything after a `#` symbol on a line is a **comment** — Python skips it entirely. Comments are notes for humans (including future-you):

```
# This line is just a note. Python ignores it.
print("But this line runs.") # a comment can also sit at the end
```

PRO TIP — COMMENT THE WHY, NOT THE WHAT

Good comments explain *why* a piece of code exists, not *what* it does line by line (the code already shows that). Write `# slow start to avoid overwhelming new users`, not `# add 1 to x`.

1.2 Variables: Labelled Boxes for Information

A variable is a name you attach to a piece of information so you can refer to it, reuse it, and change it later.

ANALOGY — LABELLED BOXES IN A WAREHOUSE

Imagine a warehouse full of identical boxes. To find anything, you stick a **label** on each box: “winter_coats”, “tax_documents”. A variable is exactly this: a **label stuck onto a value**. The label is the variable's *name*; what's inside the box is its *value*. You can open the box to read the contents, and you can swap the contents for something new while keeping the same label.

You create a variable with the **equals sign**, which here means “put the value on the right into the box named on the left”:

```
age = 15
name = "Sara"
print(name)
print(age)
```

```
Sara
15
```

WATCH OUT — = IS NOT "EQUALS" HERE

In maths, `x = 5` states a fact. In Python, `=` is an **action**: “assign”. Read `age = 15` as “let age become 15”, not “age equals 15”. This is why `x = x + 1` — which looks impossible in maths — is perfectly sensible in code: it means “take the current value in box `x`, add one, and put the result back in the same box.” (To ask whether two things are equal, you will later use `==`, with two equals signs.)

WHERE THE WORD COMES FROM — "VARIABLE"

Borrowed straight from **algebra**. In the equation $y = 2x$, the letters x and y are values that can **vary** — from the Latin *variabilis*, “changeable.” Early programmers were mathematicians, so they reused the word they already knew for “a name standing in for a value that might change.”

■ Rules and good habits for names

- Names can use letters, digits, and underscores, but **cannot start with a digit**: `score1` is fine, `1score` is an error.
- Names are **case-sensitive**: `Age` and `age` are two different boxes.
- By convention, Python names use `lower_case_with_underscores` (called *snake_case*) — e.g. `final_score`, `user_name`.
- Choose names that explain themselves. `days_until_exam` beats `d` every time.

✓ KNOWLEDGE CHECK — VARIABLES

- In plain English, what does the line `total = total + 10` instruct the computer to do?
- Why are `score` and `Score` considered different variables?
- Which of these names are invalid and why? `2nd_place`, `second_place`, `my-score`, `myScore`

1.3 The Core Data Types

Information comes in flavours. A name is text; an age is a whole number; a price has decimals; an answer to a yes/no question is true or false. Python calls these flavours **types**, and it treats each differently.

The four types you will use constantly:

Type	Python name	Holds	Example
Text	<code>str</code> (string)	Words, sentences, any characters	<code>"Lahore"</code>
Whole number	<code>int</code> (integer)	Counting numbers, no decimals	<code>42</code>
Decimal	<code>float</code>	Numbers with a fractional part	<code>3.14</code>
True/False	<code>bool</code> (boolean)	One of exactly two values	<code>True</code>

You can ask Python the type of any value with the built-in `type()` function — a great habit when something behaves unexpectedly:

```
print(type("Lahore")) # text
print(type(42))       # whole number
print(type(3.14))     # decimal
print(type(True))     # true/false
```

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

WHERE THE WORDS COME FROM

String — In early printing and telegraphy, characters were physically *strung together* in a line, like beads on a thread. Programmers kept the image: a string is a sequence of characters threaded in order.

Integer — From Latin *integer*, meaning “whole” or “untouched” (the same root as *integrity*). An integer is a number left whole, with nothing broken off into a fraction.

Float — Short for *floating-point number*. The decimal point can “float” to different positions, letting one format store both 0.001 and 1000.0. It is an engineering term about *how* the number is stored.

Boolean — Named after **George Boole**, a 19th-century English mathematician who built an entire algebra out of just *true* and *false*. His work, decades before computers existed, became the logical foundation every digital machine runs on. A whole type named in his honour.

■ Converting between types

Types are not locked. You can convert a value from one to another using `int()`, `float()`, and `str()`. This matters enormously, because user input always arrives as text:

```
price_text = "199"           # this is a string, not a number
price = int(price_text)      # convert text to a whole number
print(price + 1)             # now arithmetic works
```

```
200
```

WATCH OUT — "2" + "2" IS NOT 4

The same symbol means different things to different types. With numbers, `+` adds: `2 + 2` gives `4`. With strings, `+` **glues text together** (called *concatenation*): `"2" + "2"` gives `"22"`. This is the single most common beginner surprise. If you ever add two things and get a nonsense result, check their types with `type()`.

1.4 Operators: Doing Things With Values

Operators are the verbs of arithmetic and logic — small symbols that combine or compare values.

Arithmetic operators

Operator	Meaning	Example	Result
<code>+</code>	add	<code>7 + 3</code>	<code>10</code>
<code>-</code>	subtract	<code>7 - 3</code>	<code>4</code>
<code>*</code>	multiply	<code>7 * 3</code>	<code>21</code>
<code>/</code>	divide (always gives a float)	<code>7 / 2</code>	<code>3.5</code>
<code>//</code>	floor divide (whole part only)	<code>7 // 2</code>	<code>3</code>
<code>%</code>	modulo (the remainder)	<code>7 % 2</code>	<code>1</code>
<code>**</code>	power (exponent)	<code>2 ** 3</code>	<code>8</code>

ANALOGY — THE MODULO CLOCK

The **modulo** operator `%` gives the *remainder* after division, and the clearest picture is a **clock**. If it is 10 o'clock and you add 5 hours, it is 3 o'clock — not 15. The clock “wraps around” at 12. That wrap is modulo: `15 % 12` is `3`. Modulo is how programmers handle anything cyclical — days of the week, alternating colours, or the classic test “is this number even?” (`n % 2 == 0` means no remainder when divided by two, so it is even).

WHERE THE WORD COMES FROM — “MODULO”

From the Latin *modulus*, “a small measure.” In mathematics, working “modulo 12” means measuring numbers against a cycle of size 12 — exactly the clock. The word travelled from Roman builders' measuring units, through Gauss's number theory, into your keyboard.

Comparison operators — questions that return True or False

Operator	Asks	Example	Result
<code>==</code>	are they equal?	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	are they different?	<code>5 != 3</code>	<code>True</code>
<code>></code> <code><</code>	greater / less than	<code>5 > 3</code>	<code>True</code>
<code>>=</code> <code><=</code>	greater/less than or equal	<code>5 >= 5</code>	<code>True</code>

WATCH OUT — ONE EQUALS VS. TWO

`=` **assigns** a value (“put 5 in the box”). `==` **asks a question** (“is the box equal to 5?”). Mixing them up is the rite of passage of every programmer. If you write `if age = 18:` Python will stop you with an error — it wanted a question, you gave it a command.

Logical operators — combining questions

Use `and`, `or`, and `not` to join true/false values. `and` needs both sides true; `or` needs at least one; `not` flips the answer.

```
age = 16
has_permission = True
print(age >= 13 and has_permission)    # both true?
print(age < 13 or has_permission)      # at least one true?
print(not has_permission)               # the opposite
```

```
True
True
False
```

✓ KNOWLEDGE CHECK — TYPES & OPERATORS

1. What does `"7" + "7"` produce, and why is it different from `7 + 7`?
2. Without running it, what is `17 % 5`? What real-world idea does modulo capture?
3. Predict the result: `(10 > 2) and (3 == 4)`. Explain your reasoning.
4. Which type is named after a person, and who was he?

1.5 Strings: Working With Text

Most real programs handle text constantly — names, messages, file contents, answers from an AI. Python gives strings a rich toolkit, and it is worth knowing it well.

■ f-strings: the modern way to build text

To mix variables into text, put an `f` before the opening quote and wrap any variable in curly braces. This is called an **f-string** (formatted string) and it is the cleanest tool you have:

```
name = "Sara"
score = 92
print(f"Well done {name}, you scored {score}%!")
```

```
Well done Sara, you scored 92%!
```

PRO TIP — YOU CAN DO MATHS INSIDE THE BRACES

Anything inside `{ }` is evaluated. `f"Next year you'll be {age + 1}"` works. You can even round numbers: `f"Price: {price:.2f}"` shows two decimal places. f-strings are the Swiss-army knife of output.

■ Indexing and slicing: reaching into a string

Every character in a string has a numbered position called an **index**, and — importantly — **counting starts at zero**, not one.

ANALOGY — WHY COUNT FROM ZERO?

Think of an index as “how many steps from the start.” The first character is **zero steps** from the beginning — you are already standing on it. The second is one step away, and so on. Floors in many European buildings work the same way: the entrance is the “ground floor” (0), and the “first floor” is one flight up. Once you see indexing as *distance from the start* rather than *counting position*, zero-based counting stops feeling strange.

```
word = "Python"
print(word[0])    # first character
print(word[1])    # second character
print(word[-1])   # last character (negative counts from the end)
```

```
P
y
n
```

A **slice** grabs a range of characters using `[start:stop]`. The start is included; the stop is not — think of it as “up to but not including.”

```
word = "Python"
print(word[0:3])  # characters 0, 1, 2
print(word[2:])   # from index 2 to the end
```

```
Pyt
thon
```

■ Handy string methods

A **method** is an action attached to a value, called with a dot. Strings come with dozens; these earn their keep daily:

Method	What it does	Example → result
<code>.upper()</code>	all capitals	<code>"hi".upper()</code> → <code>"HI"</code>
<code>.lower()</code>	all lowercase	<code>"HI".lower()</code> → <code>"hi"</code>
<code>.strip()</code>	remove surrounding spaces	<code>" hi ".strip()</code> → <code>"hi"</code>
<code>.replace(a,b)</code>	swap text	<code>"cat".replace("c","b")</code> → <code>"bat"</code>
<code>.split()</code>	break into a list of words	<code>"a b c".split()</code> → <code>["a","b","c"]</code>
<code>len(s)</code>	how many characters (a function, not a method)	<code>len("hi")</code> → <code>2</code>

WATCH OUT — STRINGS CANNOT BE EDITED IN PLACE

Strings are **immutable** — once created, the characters cannot be changed. `word[0] = "J"` raises an error. Methods like `.replace()` do not change the original; they hand you a *brand-new* string. So you must capture it: `word = word.replace("P", "J")`. Forgetting to reassign is a classic source of “why didn't my change stick?”

✓ KNOWLEDGE CHECK — STRINGS

1. Given `s = "Summer"`, what is `s[0]`? What is `s[-1]`? What is `s[1:4]`?
2. Why does `s[1:4]` give three characters and not four?
3. What does it mean that strings are *immutable*, and what must you do to keep the result of `.replace()`?

1.6 Input and Decisions

A program that always does the same thing is just a fancy printout. Real programs **react** — they take input and choose what to do based on it. This is where code comes alive.

■ Getting input from the user

The `input()` function pauses the program, shows a prompt, and waits for the person to type something and press Enter. Whatever they type comes back as a **string**:

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```


WATCH OUT — INPUT IS ALWAYS TEXT

Even if the user types `25`, `input()` hands you the string `"25"`, not the number. To do maths with it you must convert: `age = int(input("Age? "))`. Forgetting this is why `"25" + 5` crashes instead of giving 30.

■ The if statement: a fork in the road**ANALOGY — THE BOUNCER AT THE DOOR**

Picture a bouncer at a venue checking IDs. “If you are 18 or older, you may enter; **else** you may not.” The bouncer evaluates a true/false question and routes you down one of two paths. An `if` statement is exactly this bouncer: it checks a condition and runs one block of code or another depending on the answer.

The structure uses a colon and — crucially — **indentation** (spaces at the start of a line) to show which lines belong to which branch:

```
age = int(input("Your age? "))

if age >= 18:
    print("Welcome in.")
elif age >= 13:
    print("You may enter with a guardian.")
else:
    print("Sorry, too young today.")
```

`elif` (short for “else if”) lets you check further conditions only if the earlier ones failed. Python checks each in order and runs the **first** branch whose condition is true, then skips the rest.

WATCH OUT — INDENTATION IS THE GRAMMAR

In most languages, blocks are marked with `{ }` braces, and indentation is just for looks. In Python, **indentation is the law**. The spaces are not decoration — they tell Python which lines are inside the `if`. Mixing tabs and spaces, or using an inconsistent number of spaces, causes an `IndentationError`. Pick four spaces per level (your editor's Tab key usually does this) and never mix.

WHERE THE WORD COMES FROM — "CODE" AND "BUG"

Code comes from the Latin *codex*, a set of laws or rules written in a book — fitting, since a program is a strict set of rules. **Bug**, meaning a fault, predates computers (Edison used it), but was cemented in 1947 when engineers found an actual moth trapped in a Harvard computer's relay. They taped it into the logbook labelled “first actual case of bug being found.” To this day, fixing errors is called *debugging*.

✓ **KNOWLEDGE CHECK — DECISIONS**

1. Why must you wrap `input()` in `int()` before comparing it to a number?
2. In an `if / elif / elif / else` chain, how many branches run at most?
3. What single thing tells Python which lines belong inside an `if` block?

1.7 Week 1 Exercises

Do these in order. Type, run, and — where it says so — deliberately break them to see the error. Solutions and hints are in **Appendix B**.

EXERCISES — FOUNDATIONS

1. **WARM-UP About me.** Make three variables — your name, your age, and your favourite subject — then print a single sentence using an f-string that uses all three.
2. **WARM-UP Temperature converter.** Store a temperature in Celsius, convert it to Fahrenheit using the formula $F = C * 9/5 + 32$, and print both nicely.
3. **CORE Rectangle stats.** Ask the user for a width and a height (remember to convert to numbers!). Print the area and the perimeter.
4. **CORE Even or odd.** Ask for a whole number and use modulo with an `if/else` to tell the user whether it is even or odd.
5. **CORE Initials.** Ask for a full name like `"ada lovelace"`. Using `.split()`, indexing, and `.upper()`, print the initials: `A.L.`
6. **STRETCH Grade reporter.** Ask for a score out of 100 and print a letter grade using an `if/elif/else` chain (`90+ → A`, `80+ → B`, `70+ → C`, `60+ → D`, otherwise `F`). Add a kind message for each.
7. **STRETCH Break it on purpose.** Take exercise 4 and remove the `int(...)` conversion. Run it, read the error message carefully, and write one sentence explaining what the error is telling you.

Week 1 Project — The Personal Greeter & Quiz

Time to combine everything. Build a small interactive program that greets the user and runs a one-question number quiz. This single script uses input, type conversion, f-strings, comparison, and decisions — the entire week in one place.

YOUR MISSION

Write a program that (1) asks the user's name and greets them by it, (2) asks how old they are and tells them how old they will be in 10 years, (3) asks them to guess a “secret number” you have stored, and (4) tells them whether their guess was too high, too low, or exactly right. Below is a complete reference solution — but **try it yourself first**, then compare.

```
# ===== The Personal Greeter & Quiz =====
secret_number = 7

# 1. Greeting
name = input("Hi! What's your name? ").strip()
print(f"Lovely to meet you, {name}.")

# 2. Age maths
age = int(input("How old are you? "))
print(f"In 10 years you'll be {age + 10}. Not so far away!")

# 3 & 4. The guessing quiz
guess = int(input("I'm thinking of a number from 1-10. Guess it: "))

if guess == secret_number:
    print(f"Incredible, {name}! You read my mind.")
elif guess < secret_number:
    print("Too low – aim higher next time.")
else:
    print("Too high – come down a little.")
```

MAKE IT YOURS — STRETCH IDEAS

Once it works, extend it: add a second guess; congratulate differently if they are within 1 of the secret; or compliment their name if it has more than 5 letters (`len(name) > 5`). Every small extension you invent yourself is worth ten you copy.

END OF WEEK 1 — WHAT YOU CAN NOW DO

You can store data, work with text and numbers, take input, and make your program choose between paths. That is the genuine core of programming — everything else is building on this. Take a breath. Next week you learn to make the computer repeat work and organise data, which is where programs start to feel powerful.

02

WEEK TWO

Structure: Building Reusable Machinery

Last week you wrote single instructions. This week you learn to multiply your power: to repeat work without rewriting it, to store collections of data in sensible containers, and to package logic into named tools you can use again and again. This is where a beginner becomes a builder.

- ▶ Repeat actions with `while` and `for` loops
- ▶ Store ordered collections in lists and tuples
- ▶ Map keys to values with dictionaries
- ▶ Track unique items with sets
- ▶ Package reusable logic into functions with parameters and return values
- ▶ Understand scope: where variables live and die
- ▶ Write elegant comprehensions to transform data in one line

2.1 Loops: Doing It Again Without Repeating Yourself

Computers are tireless. Their superpower is doing the same thing thousands of times without complaint or error. A loop is how you unleash that — you write an instruction once and tell Python how many times to run it.

WHERE THE WORDS COME FROM

Loop — A literal image of a piece of string or wire bent back to its start, forming a closed circle. Code that runs and then jumps back to its beginning traces that same circular path, so the name stuck.

Iterate — From the Latin *iterare*, “to repeat” (from *iterum*, “again”). Each single pass through a loop is called an *iteration*. You will hear “iterate over a list” constantly — it just means “go through it one item at a time.”

■ The for loop: do something for each item

A `for` loop walks through a collection, handing you one item at a time. The clearest case is looping over a range of numbers:

```
for number in range(5):
    print(f"Count: {number}")
```

```
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
```

ANALOGY — A POSTMAN ON A STREET

A `for` loop is a postman walking down a street with a stack of letters. For *each* house on the list, he does the same action: deliver a letter. He does not decide how many houses there are — the street (the collection) does. He simply visits every one in turn until the street ends.

`for house in street:` reads almost like English: “for each house in the street, do this.”

WATCH OUT — RANGE(5) GIVES 0 TO 4

`range(5)` produces five numbers, but they are **0, 1, 2, 3, 4** — it stops *before* 5, never reaching it. This matches string slicing’s “up to but not including” rule. If you want 1 to 5, write `range(1, 6)`. Off-by-one mistakes here are so common they have a nickname: *fencepost errors*.

■ The while loop: keep going until a condition changes

A `while` loop repeats *as long as* a condition stays true. Use it when you don't know in advance how many repetitions you need — like asking for input until the user gets it right:

```
password = ""
while password != "swordfish":
    password = input("Enter the password: ")
print("Access granted.")
```

WATCH OUT — THE INFINITE LOOP

A `while` loop runs forever if its condition *never* becomes false. `while True:` with no escape will freeze your program. Always ask: “what inside this loop will eventually make the condition false?” If nothing does, you have an infinite loop. (To stop a runaway program in the terminal, press `Ctrl` + `C`.)

PRO TIP — FOR VS. WHILE

Rule of thumb: use a `for` loop when you know the collection or count ahead of time (“for each student”, “repeat 10 times”). Use a `while` loop when you are waiting for a condition to change (“until the user quits”, “while the game is not over”).

✓ KNOWLEDGE CHECK — LOOPS

1. How many numbers does `range(3)` produce, and what are they exactly?
2. You want to greet 100 users you already have in a list. Which loop fits better, `for` or `while`, and why?
3. What is an infinite loop, and what is the one question that prevents it?

2.2 Collections: Containers for Many Values

A single variable holds one thing. But real data comes in groups: a class of students, a shopping list, a phone book. Python offers four core containers, each shaped for a different job.

■ Lists — an ordered, changeable sequence

ANALOGY — A SHOPPING LIST ON PAPER

A list is exactly a handwritten shopping list. The items are in an **order** (top to bottom), you can **add** to the end, **cross items off**, and **change** an entry. You can have duplicates (“milk” twice). Python lists work the same way — ordered, editable, and happy to repeat.

Lists use square brackets. Items are reached by index (starting at 0, just like strings):

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])           # apple
fruits.append("date")      # add to the end
fruits[1] = "blueberry"    # change an item
print(fruits)              # the whole list
print(len(fruits))         # how many items
```

```
apple
['apple', 'blueberry', 'cherry', 'date']
4
```

Lists shine inside `for` loops — this is the most common pattern in all of Python:

```
for fruit in fruits:
    print(f"I like {fruit}.")
```

■ Tuples — an ordered sequence that cannot change

A tuple looks like a list but uses round brackets and is **immutable** — once created, it is locked. Use a tuple for a group of values that belong together and should never change, like a coordinate or a date.

```
point = (3, 5)             # an (x, y) coordinate
print(point[0])            # 3
# point[0] = 9 would raise an error – tuples are frozen
```

WHERE THE WORD COMES FROM — "TUPLE"

Mathematicians needed words for ordered groups: a pair, a triple, a quadruple, a quintuple, a sextuple... The shared ending is **-tuple** (from the Latin suffix meaning “-fold”). When they needed a general word for “an ordered group of any size,” they simply kept the ending and dropped the front: a **tuple**. So a tuple is, quite literally, the leftover piece of “quin-tuple.”

■ Dictionaries — look things up by name, not position

ANALOGY — A REAL DICTIONARY

In a paper dictionary you don't find a word by its *position* (“the 4,102nd word”); you find it by the **word itself**, which leads you to its **definition**. A Python dictionary works identically: each entry is a **key** (the word you look up) paired with a **value** (what you get back). Phone contacts are a perfect example — you look up a person's *name* (key) to get their *number* (value).

Dictionaries use curly braces with `key: value` pairs:

```
ages = {
    "Sara": 15,
    "Ali": 17,
    "Zoya": 14
}
print(ages["Ali"])           # look up by key → 17
ages["Bilal"] = 16          # add a new pair

for name, age in ages.items():
    print(f"{name} is {age}")
```

```
17
Sara is 15
Ali is 17
Zoya is 14
Bilal is 16
```

PRO TIP — DICTIONARIES ARE EVERYWHERE

When you call an AI or any web service in Week 4, the data that comes back is almost always a dictionary — keys like **"role"** and **"content"** pointing to values. Master dictionaries now and the AI capstone will feel natural.

■ Sets — a bag of unique items

A set stores items with **no duplicates** and **no order**. Its superpower is instantly removing repeats and answering “is this in here?” quickly.

```
numbers = [1, 2, 2, 3, 3, 3]
unique = set(numbers)
print(unique)           # {1, 2, 3} – duplicates gone
```

WHERE THE WORD COMES FROM — "SET"

Straight from mathematics. A **set** in maths (formalised by Georg Cantor in the 1870s) is a collection of distinct objects — no element appears twice, and order does not matter. Python's set obeys exactly those rules, borrowing the concept whole.

■ Choosing the right container

Container	Ordered?	Changeable?	Duplicates?	Reach for it when...
list <code>[]</code>	Yes	Yes	Yes	order matters and the collection grows or changes
tuple <code>()</code>	Yes	No	Yes	a fixed group that must never change
dict <code>{k:v}</code>	Insertion	Yes	Keys unique	you look things up by name/label
set <code>{ }</code>	No	Yes	No	you need uniqueness or fast membership tests

✓ KNOWLEDGE CHECK — COLLECTIONS

1. You are storing a list of GPS waypoints that must never be altered. List or tuple? Why?
2. You want to count how many *distinct* words appear in an essay. Which container makes this trivial?
3. In a dictionary, what is the difference between a key and a value? Give a real-world pair.
4. What does `ages.items()` let you do inside a `for` loop?

2.3 Functions: Naming and Reusing Logic

A function is a named, reusable block of instructions. Write it once, call it by name as often as you like. Functions are the single most important idea for keeping programs understandable as they grow — and they are the doorway to Week 3's objects.

ANALOGY — A KITCHEN APPLIANCE

Think of a **blender**. You don't care how its motor works; you put fruit **in**, press a button, and get a smoothie **out**. A function is your own custom appliance: it takes **inputs** (the fruit), does some hidden work, and hands back an **output** (the smoothie). Once built, you reuse it endlessly without thinking about the internals. Good programs are built from many small, reliable appliances.

You define a function with `def`, list its inputs (called **parameters**) in brackets, and use `return` to send a result back:

```
def area_of_rectangle(width, height):
    result = width * height
    return result

# "call" the function by name, passing in arguments
room = area_of_rectangle(4, 5)
print(f"The room is {room} square metres.")
```

The room is 20 square metres.

WHERE THE WORD COMES FROM — "FUNCTION" & "PARAMETER"

Function — From mathematics: $f(x)$ is a machine that takes an input x and produces an output. The word traces to the Latin *functio*, “performance” or “execution.” A programming function is the same idea made concrete: feed it values, it performs and returns.

Parameter — From Greek *para* (“beside”) + *metron* (“measure”): “a measure set alongside.” Parameters are the named slots beside the function that receive its inputs.

TWO WORDS TO KEEP STRAIGHT: PARAMETER VS. ARGUMENT

The **parameter** is the name in the definition (`width`). The **argument** is the actual value you pass in when calling (`4`). Parameters are the empty labelled cups; arguments are the drinks you pour into them.

■ Default values and named arguments

Parameters can have **defaults**, used when the caller doesn't supply a value — making functions flexible:

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Sara"))           # uses the default
print(greet("Ali", "Good morning")) # overrides it
```

Hello, Sara!
Good morning, Ali!

WATCH OUT — PRINT IS NOT RETURN

This trips up everyone. `print()` shows a value on screen but throws it away. `return` hands the value back so the rest of your program can use it. A function that `print`s its answer but doesn't `return` it cannot be used in `total = add(2, 3)` — `total` would be empty (`None`). Rule: compute and **return**; print only when you actually want something on screen.

■ Why functions matter so much

- **Don't Repeat Yourself (DRY).** Logic written once lives in one place. Fix a bug there and it's fixed everywhere.
- **Readability.** `calculate_tax(salary)` tells a story; ten lines of raw arithmetic do not.
- **Testability.** A small function with clear inputs and outputs is easy to check.

2.4 Scope: Where Variables Live and Die

A variable created inside a function exists only inside that function. The moment the function finishes, those variables vanish. This boundary is called **scope**, and understanding it prevents a whole category of confusing bugs.

ANALOGY — A CONVERSATION IN A PRIVATE ROOM

Imagine each function is a **soundproof room**. Whatever is said inside (its *local* variables) cannot be heard outside. When the meeting ends, the room is emptied and everything written on its whiteboard is wiped. The hallway outside is the *global* space — visible to everyone, but it is good manners to keep most of your work inside rooms rather than shouting everything into the hallway.

```
def make_total():
    subtotal = 100      # local: lives only in this room
    return subtotal

print(make_total())    # 100 — fine
# print(subtotal)      would error: "subtotal is not defined" out here
```

WHERE THE WORD COMES FROM — "SCOPE"

From the Greek *skopos*, “target” or “range of vision” (the same root as *telescope* and *microscope*). A variable's scope is the region of code where it is *visible* — how far you can “see” it from.

PRO TIP — PASS IN, RETURN OUT

Resist the urge to reach out and grab global variables from inside a function. Instead, **pass** what the function needs in as parameters and **return** its result. This keeps each function self-contained and predictable — you can understand it without reading the rest of the program.

2.5 Comprehensions: Elegant Transformations

A list comprehension builds a new list from an existing one in a single readable line. It is Python's signature feature — once it clicks, you will reach for it constantly.

Compare the long way and the comprehension way. Both square every number in a list:

```
# The long way
squares = []
for n in range(1, 6):
    squares.append(n * n)

# The comprehension way – same result, one line
squares = [n * n for n in range(1, 6)]
print(squares)
```

```
[1, 4, 9, 16, 25]
```

Read it right-to-left-ish as a sentence: “give me `n * n`, for each `n` in the range.” You can add a condition to filter:

```
evens = [n for n in range(1, 11) if n % 2 == 0]
print(evens) # [2, 4, 6, 8, 10]
```

WATCH OUT — READABILITY FIRST

Comprehensions are elegant, but a comprehension crammed with multiple conditions and nested loops becomes a puzzle. If you can't read yours aloud as a clear sentence, write it as a normal `for` loop instead. Clever code that no one can read — including future-you — is a liability, not a flex.

✓ KNOWLEDGE CHECK — FUNCTIONS, SCOPE & COMPREHENSIONS

1. What is the difference between a *parameter* and an *argument*?
2. Why can a function that uses `print()` but not `return` cause `x = my_func()` to leave `x` empty?
3. What does “scope” mean, and why can't you read a function's local variable from outside it?
4. Rewrite this as a list comprehension: a list of the lengths of each word in `["hi", "world", "ok"]`.

2.6 Week 2 Exercises

EXERCISES — STRUCTURE

1. **WARM-UP Countdown.** Use a `for` loop and `range` to print a countdown from 10 to 1, then print “Lift off!”
2. **WARM-UP Sum a list.** Given `prices = [4, 9, 2, 7]`, use a loop to compute and print the total. (Then discover the built-in `sum()` and compare.)
3. **CORE Word lengths.** Write a function `longest(words)` that takes a list of words and returns the longest one. Test it on a few lists.
4. **CORE Vowel counter.** Write a function `count_vowels(text)` that returns how many vowels are in a string. (Hint: loop over the characters; check `if char in "aeiou"`.)
5. **CORE Phone book.** Build a dictionary of three names → numbers. Then write a loop that prints each entry as `"Name: number"`. Add a new contact and reprint.
6. **CORE Unique words.** Given a sentence, use `.split()` and a `set` to print how many *distinct* words it contains.
7. **STRETCH FizzBuzz.** The classic. For numbers 1 to 30, print “Fizz” if divisible by 3, “Buzz” if divisible by 5, “FizzBuzz” if both, otherwise the number. (Modulo is your friend.)
8. **STRETCH Grade book function.** Write `average(scores)` that returns the mean of a list of numbers, then a function `letter_grade(avg)` that returns A/B/C/D/F. Combine them: feed a student's scores in and print their letter grade.

Week 2 Project — The To-Do List Manager

Combine loops, lists, functions, and a `while` loop into a small interactive app that genuinely works. The user can add tasks, view them, mark them done, and quit — the same menu pattern behind countless real programs.

YOUR MISSION

Build a menu-driven to-do manager. Tasks live in a list. The program loops, showing a menu, until the user chooses to quit. Each feature is a small function. Reference solution below — build your own first.

```
# ===== The To-Do List Manager =====
tasks = []

def show_tasks():
    if not tasks:
        print("  (no tasks yet)")
    for i, task in enumerate(tasks, start=1):
        print(f"  {i}. {task}")

def add_task(text):
    tasks.append(text)
    print(f"Added: {text}")

while True:
    print("\n--- TO-DO MENU ---")
    print("1) View  2) Add  3) Quit")
    choice = input("Choose: ").strip()

    if choice == "1":
        show_tasks()
    elif choice == "2":
        new = input("New task: ").strip()
        add_task(new)
    elif choice == "3":
        print("Goodbye! Stay productive.")
        break
    else:
        print("Please choose 1, 2, or 3.")
```

NEW THINGS IN THIS PROJECT

`enumerate(tasks, start=1)` hands you both the position and the item, so you can number tasks 1, 2, 3. `break` immediately exits the nearest loop — the clean way to leave a `while True`. `not tasks` is `True` when the list is empty (empty collections count as false in Python — a tidy idiom).

MAKE IT YOURS — STRETCH IDEAS

Add a “mark done” option that removes a task by its number (`tasks.pop(number - 1)`). Show a count of remaining tasks. Refuse to add an empty task. Each addition teaches you something the next project will reuse.

END OF WEEK 2 — WHAT YOU CAN NOW DO

You can repeat work, organise data into the right container, and bundle logic into reusable functions. You have built a real interactive app. Crucially, you now think about *data* (the tasks list) and *behaviour* (the functions that act on it) as separate things. Next week, you'll discover that bundling data and behaviour *together* is the powerful idea called object-oriented programming — and your to-do app will be the perfect thing to rebuild with it.

03

WEEK THREE

Modelling the World: Object-Oriented Programming

This is the summit of the climb. So far you have had data in one hand and functions in the other. This week you learn to fuse them – to create your own data types that carry both their information and the actions that belong to them. This is how nearly all serious software is designed, and it is the way professional programmers think.

- ▶ Understand why OOP exists and the problem it solves
- ▶ Define your own classes and create objects from them
- ▶ Use `__init__` and `self` confidently
- ▶ Apply the four pillars: encapsulation, inheritance, polymorphism, abstraction
- ▶ Give your objects natural behaviour with dunder (magic) methods
- ▶ Design a small class hierarchy of your own

3.1 The Big Idea: Why Objects?

Look around you. The world is made of **things** — objects that have *properties* and can *do things*. A dog has a name and a breed (properties), and it can bark and fetch (actions). Object-oriented programming is simply the idea of modelling your program the same way the world is built.

In Week 2 you kept data and behaviour apart: a `tasks` list over here, an `add_task` function over there. That works for small programs, but as a program grows, loose data and loose functions drift apart, and it becomes hard to know which functions are *allowed* to touch which data. OOP fixes this by **bundling related data and the functions that act on it into a single unit** — an object.

ANALOGY — NOUNS AND VERBS

If a program were a sentence, the **objects are the nouns** (a *player*, an *account*, a *message*) and the **methods are the verbs** they can perform (the player *jumps*, the account *withdraws*, the message *sends*). OOP asks you to first identify the nouns in your problem, then ask what each one knows (its data) and what it can do (its behaviour). Suddenly your code reads like a description of the real situation.

■ Classes and objects: the blueprint and the building

ANALOGY — THE BLUEPRINT AND THE HOUSES

A **class** is a *blueprint*. A blueprint is not a house — you cannot live in it — but it describes exactly how to build one: where the doors go, how many rooms. From one blueprint you can build **many actual houses** (objects), each identical in structure but with its own details — this one painted blue, that one with the lights on. The blueprint is written once; the houses (called **instances**) are created from it as often as you need.

WHERE THE WORDS COME FROM

Class — From the Latin *classis*, a group or division of citizens (Roman society was sorted into classes). In science, Linnaeus used “class” to group living things sharing a structure — mammals, birds. Programming borrowed it for the same reason: a class groups objects that share the same structure and behaviour.

Object — From the Latin *obiectum*, “a thing thrown before” the mind — a thing you can think about and act upon. In code, an object is exactly that: a concrete thing your program holds and manipulates.

Instance — From Latin *instantia*, “an example or a case of.” Each house built from the blueprint is an *instance* of it — one concrete example of the general class.

■ Your first class

Let's model a dog. The class is the blueprint; each dog you make is an instance:

```
class Dog:
    def __init__(self, name, breed):
        self.name = name          # store data ON this dog
        self.breed = breed

    def bark(self):
        return f"{self.name} says Woof!"

# Build two actual dogs from the one blueprint
rex = Dog("Rex", "Labrador")
bella = Dog("Bella", "Poodle")

print(rex.bark())                # Rex says Woof!
print(bella.breed)               # Poodle
```

```
Rex says Woof!
Poodle
```

VOCABULARY, ALL IN ONE PLACE

A **class** is the blueprint (`Dog`). An **object** or **instance** is a thing built from it (`rex`). The data stored on an object (`name` , `breed`) are its **attributes**. The functions defined in a class (`bark`) are its **methods**. Note the convention: class names use **CapitalisedWords** , so you can spot a blueprint at a glance.

3.2 Making Sense of `__init__` and `self`

Two pieces of that `Dog` class look strange at first: the oddly-named `__init__` method and the `self` that appears everywhere. They are the heart of how objects work, and they are simpler than they look.

■ `__init__` — the setup ritual

When you write `Dog("Rex", "Labrador")`, Python builds a fresh, empty dog and then immediately runs a special method called `__init__` to **set it up** — to give it its starting attributes. You never call `__init__` yourself; Python calls it automatically the moment an object is born.

ANALOGY — THE FACTORY SETUP LINE

`__init__` is the **setup station at the end of a factory line**. A blank product rolls off the line (the new object), and the setup station stamps it with its specific details — this phone gets *this* serial number, *this* colour. By the time the object leaves `__init__`, it is fully prepared for use. The name is short for **initialise**: “to set up for the first time.”

WHERE THE NAME COMES FROM — "DUNDER"

Methods wrapped in **double underscores** — `__init__`, `__str__` — are nicknamed **dunder** methods (a blend of “double underscore”). The underscores signal “this is special to Python, not an everyday name you invented.” Python calls these behind the scenes at the right moments.

 self — “this particular object”

Here is the key insight. One blueprint makes many objects, but each method is written only once. So when `bark()` runs, how does it know *which* dog's name to use — Rex's or Bella's? The answer is `self`: when you call `rex.bark()`, Python quietly passes `rex` in as `self`. Inside the method, `self` means “**the specific object this was called on.**”

ANALOGY — “MYSELF”

Picture a recipe card that says: “add *your* name to the label.” The card is generic — written once — but each cook reads “your” as referring to *themselves*. `self` is the object saying “me, the one this is happening to.” `self.name` means “my name”. When Rex runs `bark`, `self` is Rex, so `self.name` is “Rex”. When Bella does, `self` is Bella.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner      # my owner
        self.balance = balance  # my balance

    def deposit(self, amount):
        self.balance = self.balance + amount  # change MY balance
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient funds."
        self.balance = self.balance - amount
        return self.balance

acct = BankAccount("Sara", 100)
acct.deposit(50)           # balance now 150
print(acct.withdraw(200))  # Insufficient funds.
print(acct.balance)       # 150
```

```
Insufficient funds.
150
```

WATCH OUT — FORGETTING SELF

Two near-universal beginner errors: (1) leaving `self` out of a method's definition — every instance method must list `self` as its first parameter. (2) Forgetting `self.` when accessing an attribute — writing `balance` instead of `self.balance` inside a method makes Python look for a *local* variable that doesn't exist. If you get `NameError: 'balance' is not defined` inside a class, a missing `self.` is the usual culprit.

✓ KNOWLEDGE CHECK — CLASSES, INIT & SELF

1. In one sentence each: what is a class, and what is an object (instance)?
2. When does `__init__` run, and what is its job?
3. If one `bark()` method is shared by every dog, how does it know whether to say “Rex” or “Bella”?
4. What is the difference between an *attribute* and a *method*?

3.3 The Four Pillars of OOP

Object-oriented programming rests on four big ideas. They sound intimidating, but each is a simple, intuitive principle once you meet its analogy. Together they are why OOP scales to enormous programs.

■ Pillar 1 — Encapsulation: hide the wiring

Encapsulation means bundling data with the methods that use it, and **hiding the internal details** so the outside world interacts only through a clean, safe interface.

ANALOGY — DRIVING A CAR

To drive a car you use the steering wheel, pedals, and gear stick. You do **not** reach into the engine and adjust fuel injection by hand — and that is a good thing. The complex machinery is sealed under the bonnet; you are given a simple, safe set of controls. Encapsulation is exactly this: an object exposes a few sensible methods (the controls) and hides its messy internals (the engine), so you cannot accidentally break it.

WHERE THE WORD COMES FROM — "ENCAPSULATE"

From the Latin *capsula*, “a little box” (the same root as a medicine *capsule*). A pill capsule wraps its contents in a protective shell so you handle the outside, never the powder within. Encapsulation wraps an object's data in a protective shell of methods.

In Python, a leading underscore signals “this is internal — please don't touch from outside.” The bank account already encapsulates well: you change the balance only through `deposit` and `withdraw`, which can *enforce rules* (no overdrafts). If anyone could write `acct.balance = -999` directly, the rules would mean nothing.

■ Pillar 2 — Inheritance: build on what exists

Inheritance lets a new class take on all the attributes and methods of an existing one, then add or change a few. The original is the **parent** (or base) class; the new one is the **child** (or derived) class.

ANALOGY — A FAMILY TREE

A child inherits traits from a parent — eye colour, surname — and then develops their own. In code, a `SavingsAccount` can inherit everything a `BankAccount` already does (deposit, withdraw) and add something new (earn interest). You don't rewrite deposit and withdraw — you inherit them, free, and build on top. This is the great labour-saver of OOP.

WHERE THE WORD COMES FROM — "INHERIT"

From the Latin *hereditare*, “to receive as an heir.” Just as an heir receives property without earning it from scratch, a child class receives its parent's methods and attributes automatically.

```
class SavingsAccount(BankAccount):    # inherits from BankAccount
    def __init__(self, owner, balance=0, rate=0.05):
        super().__init__(owner, balance)    # run the parent's setup
        self.rate = rate                    # add something new

    def add_interest(self):
        self.balance += self.balance * self.rate
        return self.balance

s = SavingsAccount("Ali", 1000)
s.deposit(500)    # inherited from BankAccount — works for free!
print(s.add_interest())    # 1575.0 — its own new ability
```

1575.0

PRO TIP — `SUPER()`

`super()` means “the parent class.” Calling `super().__init__(...)` runs the parent's setup so you don't duplicate it, then you add only what's new. Reuse the parent; extend, don't rewrite.

■ Pillar 3 — Polymorphism: one command, many forms

Polymorphism means different objects can respond to the **same command in their own way**. You call `.area()` on a shape; a circle computes it one way, a square another — but you, the caller, just say `.area()` and trust each object to do the right thing.

WHERE THE WORD COMES FROM — "POLYMORPHISM"

Pure Greek: *poly* (“many”) + *morphē* (“form”) = “many forms.” The same word, the same call, takes many forms depending on the object receiving it. (Water is a real-world polymorph: ice, liquid, steam — one substance, many forms.)

```
class Circle:
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2

class Square:
    def __init__(self, side): self.side = side
    def area(self): return self.side ** 2

shapes = [Circle(5), Square(4), Circle(2)]
for shape in shapes:
    print(shape.area())    # same call – each computes its own way
```

```
78.53975
16
12.56636
```

WHY THIS IS POWERFUL

The loop doesn't care what shape it holds. It says `.area()` and each object answers correctly. You can invent a new `Triangle` class tomorrow with its own `.area()`, drop it into the list, and the loop works unchanged. Polymorphism lets your code accept things that don't even exist yet, as long as they speak the same “language” of method names.

■ Pillar 4 — Abstraction: define what, not how

Abstraction means focusing on *what* an object does while ignoring *how* it does it. You design a simple concept (“a shape has an area”) and let each class fill in the details. It is the principle that makes encapsulation, inheritance, and polymorphism worth having.

ANALOGY — A TV REMOTE

A remote control gives you abstract commands: *volume up*, *channel next*, *power*. You think purely in terms of *what* you want, never the infrared signals and circuitry that make it happen. Abstraction is designing those clean, intention-level commands and hiding the “how” entirely. Good classes feel like good remotes: their methods describe intentions, not mechanics.

WHERE THE WORD COMES FROM — “ABSTRACT”

From the Latin *abstrahere*, “to draw away.” You *draw away* from the messy specifics to see the general shape underneath. To abstract is to step back far enough that only the essential idea remains.

✓ **KNOWLEDGE CHECK — THE FOUR PILLARS**

1. Name the four pillars. For each, give the one-line analogy that captures it.
2. Why is letting outside code write `account.balance = -999` a violation of encapsulation?
3. What does `super().__init__()` do, and why is it useful in a child class?
4. A loop calls `.area()` on a mixed list of shapes and each responds correctly. Which pillar is this, and what does its name literally mean?

3.4 Dunder Methods: Teaching Objects Natural Behaviour

You met `__init__`. Python has a whole family of these double-underscore methods that let your objects behave like built-in types — printing nicely, being compared, having a length. Define them and your objects feel native.

Dunder method	Python calls it when you...
<code>__init__</code>	create the object — sets it up
<code>__str__</code>	use <code>print()</code> or <code>str()</code> — the friendly text
<code>__repr__</code>	inspect it in the REPL — the developer's text
<code>__eq__</code>	compare two objects with <code>==</code>
<code>__len__</code>	call <code>len()</code> on it

Without `__str__`, printing an object gives an ugly memory address. With it, you control exactly how it appears:

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f'{self.title}' ({self.pages} pages)"

    def __len__(self):
        return self.pages          # len(book) now means its page count

b = Book("Python Playbook", 72)
print(b)          # 'Python Playbook' (72 pages) - thanks to __str__
print(len(b))     # 72 - thanks to __len__
```

'Python Playbook' (72 pages)
72

PRO TIP — ALWAYS ADD `__str__`

For any class you build, write a `__str__` early. It makes debugging dramatically easier — every `print` of your object becomes informative instead of cryptic. It is the single highest-value dunder for a beginner.

■ Class variables vs. instance variables

An **instance variable** (set with `self.`) is unique to each object — each dog has its own name. A **class variable** is shared by *all* instances — useful for things every object should agree on, like a species or a counter.

```
class Dog:
    species = "Canis familiaris"  # class variable: shared by all dogs

    def __init__(self, name):
        self.name = name         # instance variable: unique per dog

print(Dog("Rex").species)       # Canis familiaris
print(Dog("Bella").species)     # Canis familiaris (same for all)
```

■ Properties: encapsulation made elegant

Sometimes you want an attribute that is read like data but secretly runs code — for validation, say. The `@property` decorator does this. It lets you protect an attribute (true encapsulation) while still accessing it with clean dot syntax:

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius  # underscore: "internal"

    @property
    def fahrenheit(self):        # computed on demand, looks like data
        return self._celsius * 9/5 + 32

t = Temperature(25)
print(t.fahrenheit)            # 77.0 — accessed like an attribute, no ()
```

✓ **KNOWLEDGE CHECK — DUNDER & BEYOND**

1. What does defining `__str__` change about how your object behaves with `print()` ?
2. What is the difference between a class variable and an instance variable? Give an example of each for a `Student` class.
3. Why might you use `@property` instead of a plain attribute?

3.5 Week 3 Exercises

EXERCISES — OBJECT-ORIENTED THINKING

1. **WARM-UP Rectangle class.** Build a `Rectangle` class with `width` and `height` attributes and methods `area()` and `perimeter()` . Create two and print their stats.
2. **WARM-UP Add a str.** Give your `Rectangle` a `__str__` so that printing it shows `"Rectangle 4x5 (area 20)"` .
3. **CORE Student grades.** Build a `Student` class storing a name and a list of scores. Add a method `average()` and a method `add_score(s)` . Create a student, add scores, print the average.
4. **CORE Inheritance.** Make a `Animal` base class with a `name` and a `speak()` method that returns `"..."` . Make `Cat` and `Dog` subclasses that override `speak()` to return “Meow” and “Woof”. Put several in a list and loop, calling `speak()` on each (this is polymorphism in action).
5. **CORE Encapsulated counter.** Build a `Counter` class with an internal `_count` starting at 0, an `increment()` method, and a `value` property that returns the count. Make it impossible to set the count to a negative number.
6. **STRETCH Library system.** Build a `Book` class and a `Library` class. The library holds a list of books, can `add_book()` , `list_books()` , and `find_by_author(name)` . Use `__str__` on `Book` for nice listing.
7. **STRETCH Rebuild Week 2's to-do app with OOP.** Create a `Task` class (with `done` status) and a `ToDoList` class that holds tasks and has methods to add, complete, and display them. Notice how much cleaner the design feels than loose lists and functions.

■ Week 3 Project — A Text Adventure with Objects

Model a tiny game world using classes. This brings every pillar together: a `Player` (encapsulated health and inventory), `Item` objects, and rooms — all things with data and behaviour.


```

class Player:
    def __init__(self, name):
        self.name = name
        self.health = 100
        self.inventory = []

    def take_damage(self, amount):
        self.health = max(0, self.health - amount)    # never below 0

    def pick_up(self, item):
        self.inventory.append(item)
        return f"{self.name} picked up {item}."

    def __str__(self):
        return f"{self.name} | HP: {self.health} | Bag: {self.inventory}"

hero = Player("Aria")
print(hero.pick_up("torch"))
hero.take_damage(30)
print(hero)

```

```

Aria picked up torch.
Aria | HP: 70 | Bag: ['torch']

```

MAKE IT YOURS — STRETCH IDEAS

Add a **Monster** class that can **attack(player)**. Add a **heal()** method capped at 100. Build a **Room** class with a description and a list of items, and let the player **move** between rooms held in a dictionary. You now have the bones of a real game — and every piece is a clean, self-contained object.

END OF WEEK 3 — YOU ARE NOW A DESIGNER

You can model a problem as objects with data and behaviour, and you have the four pillars to organise them. This is the way professionals think about software. One week remains: connecting your programs to files, handling errors gracefully, and reaching out to the wider world — culminating in your own AI-powered application.

04

WEEK FOUR

Reaching Out: Files, Errors & Your First AI App

Your programs have lived in a sealed room. This week you open the doors. You will learn to borrow code other people wrote, to save and load files so your work survives, to catch errors before they crash everything – and finally to connect to a real artificial-intelligence service and build an app that thinks. This is where it stops feeling like an exercise and starts feeling like power.

- ▶ Import and use modules — the standard library and beyond
- ▶ Install packages with `pip` and isolate projects with virtual environments
- ▶ Read from and write to files using `with`
- ▶ Handle errors gracefully with `try / except`
- ▶ Store and read structured data as JSON
- ▶ Understand what an API is and call the OpenAI API safely
- ▶ Build a complete, object-oriented AI study application

4.1 Standing on Shoulders: Modules & Packages

No programmer writes everything from scratch. The real skill is knowing what already exists and reaching for it. Python ships with a huge **standard library** — hundreds of ready-made tools — and beyond it lies a universe of free packages. To use any of them, you `import`.

ANALOGY — THE WELL-STOCKED KITCHEN

You *could* grind your own flour and churn your own butter, but a good cook keeps a pantry of staples and reaches for them. A **module** is a jar in the pantry — a file full of ready-made functions someone already wrote and tested. `import math` is you taking the “math” jar off the shelf. You didn't write square root; you just opened the jar and used it.

WHERE THE WORDS COME FROM

Module — From the Latin *modulus*, “a small measure”, the same root as “model” and “mould”. A module is a self-contained, standard-sized piece designed to slot together with others — exactly like modular furniture.

Import — Latin *importare*, “to carry in” (*in-* + *portare*, to carry — the same *portare* in “portable” and “porter”). When you `import`, you literally carry code in from another file into your own.

Library — Latin *librarium*, “a chest for books” (from *liber*, book). A code library is exactly that: a collection of ready-to-read works you borrow from rather than write yourself.

```
# Three ways to import — all useful
import math                # grab the whole jar; use as math.sqrt
print(math.sqrt(16))       # 4.0
print(math.pi)             # 3.141592653589793

from random import randint # take just one tool out of the jar
print(randint(1, 6))       # a dice roll, e.g. 4

import datetime as dt      # give the jar a shorter nickname
print(dt.date.today())    # 2026-06-24
```

```
4.0
3.141592653589793
4
2026-06-24
```

PRO TIP — A TOUR OF THE STANDARD LIBRARY

A few jars worth knowing by name: `math` (square roots, trig, constants), `random` (dice, shuffles, random choices), `datetime` (dates and times), `os` (talk to the operating system — we use it for secrets later), `json` (save structured data), and `statistics` (mean, median). Type `import this` for a hidden poem about Python's philosophy — a genuine Easter egg in the language.

■ Modules, packages, and your own files

A single `.py` file is a **module**. A folder of related modules bundled together is a **package**. Once you save your own file — say `helpers.py` — you can `import helpers` from another file in the same folder and use its functions. That is all a module ever is: a file you can borrow from.

✓ KNOWLEDGE CHECK — MODULES

1. What is the difference between `import math` and `from math import sqrt`? How does the way you *call* the function change?
2. In plain words, what is the difference between a module and a package?
3. Why might you write `import datetime as dt` instead of just `import datetime`?

4.2 Borrowing from the World: pip & Virtual Environments

The standard library is generous, but the real treasure is outside it. Hundreds of thousands of free packages live on the **Python Package Index** (PyPI). The tool that fetches and installs them is `pip`.

WHERE THE WORDS COME FROM

pip — A playful recursive acronym, “Pip Installs Packages” (programmers love jokes that refer to themselves). A “pip” is also a tiny seed — fitting, since each package is a small seed you plant into your project.

Package — From Middle Dutch *pak*, a bundle tied for carrying. A code package is a bundle of modules wrapped up so you can carry it into your project in one move.

You run `pip` from your terminal — not from inside Python. To install the OpenAI library we will need this week:

```
# In your terminal (not the Python prompt):
pip install openai

# See what you have installed:
pip list

# Install an exact version:
pip install openai==1.40.0
```

■ Why virtual environments matter

ANALOGY — A SEPARATE TOOLBOX PER JOB

Imagine a builder who threw every tool from every job into one giant pile. Project A needs an old hammer; Project B needs the new one; they conflict and chaos follows. A **virtual environment** is a clean, dedicated toolbox for one project. Project A gets its own box with exactly the tools and versions it needs; Project B gets its own. Nothing leaks between them, and you can throw a box away without affecting anything else.

Creating one is two commands. The `venv` module (built in) makes the box; `activate` steps into it.

```
# Create a virtual environment named .venv
python -m venv .venv

# Activate it – Windows:
.venv\Scripts\activate
# Activate it – macOS / Linux:
source .venv/bin/activate

# Now pip installs land ONLY in this box:
pip install openai

# When you are done:
deactivate
```

PRO TIP — THE FREEZE-AND-RESTORE TRICK

Run `pip freeze > requirements.txt` to write a list of every package and version your project uses into a file. Anyone (including future you) can then recreate the exact toolbox with `pip install -r requirements.txt`. This one habit is what makes a project work on someone else's computer instead of only yours.

WATCH OUT — ACTIVATE BEFORE YOU INSTALL

If you forget to activate your environment, `pip install` dumps the package into your system-wide Python, recreating the big-messy-pile problem. The giveaway that you are inside an environment: most terminals show `(.venv)` at the start of the prompt line. No `(.venv)`? Activate first.

✓ KNOWLEDGE CHECK — PIP & VENV

1. Where do you run `pip` — inside the Python prompt, or in your terminal?
2. In one sentence, what problem does a virtual environment solve?
3. What does `requirements.txt` contain, and why is it so useful when sharing a project?

4.3 Making Work Survive: Reading & Writing Files

Every program you have written so far forgets everything the moment it ends. Close it, and the to-do list is gone. Files are how a program **remembers** — how data outlives the run that created it.

ANALOGY — MEMORY VERSUS THE FILING CABINET

Variables live in your computer's memory (RAM), which is like your desk: fast to reach, but swept completely clean every night. A **file** on disk is the filing cabinet: slower to open, but its contents are still there tomorrow, next week, after a reboot. Writing to a file is filing a page away; reading is pulling it back out.

■ The `with` statement: open, use, and always close

To use a file you **open** it, do your work, then **close** it. Forgetting to close can lose data or lock the file. Python's `with` statement closes it for you automatically — even if an error strikes midway — so it is always the right way.

```
# WRITING — "w" creates the file (or overwrites it)
with open("diary.txt", "w") as f:
    f.write("Today I learned about files.\n")
    f.write("They make my programs remember.\n")

# READING — "r" reads an existing file
with open("diary.txt", "r") as f:
    contents = f.read()
print(contents)

# APPENDING — "a" adds to the end without erasing
with open("diary.txt", "a") as f:
    f.write("PS: appending keeps the old lines.\n")
```

```
Today I learned about files.
They make my programs remember.
```

WHERE THE WORDS COME FROM

File — From the Latin *filum*, “a thread”. Clerks once stored documents by threading them onto a string or wire to keep them in order — “on file”. The digital file inherited the name and the idea: an ordered set of contents kept together.

Context manager — `with` creates a context (Latin *contexere*, “to weave together”) — a woven-together setup-and-cleanup wrapper. It guarantees the “close the file” cleanup runs no matter what.

WATCH OUT — "w" ERASES WITHOUT ASKING

Opening a file in `"w"` (write) mode **wipes its entire contents instantly**, before you write a single thing. If you mean to add to a file rather than replace it, you want `"a"` (append). Mixing these up is a classic way to delete an afternoon's work.

PRO TIP — READ LINE BY LINE

For big files, looping over the file object reads one line at a time without loading the whole thing into memory: `for line in f: .strip()` to drop the trailing newline on each line.

✓ KNOWLEDGE CHECK — FILES

1. Why is `with open(...)` preferred over calling `open()` and `close()` yourself?
2. What is the crucial difference between opening a file in `"w"` mode and `"a"` mode?
3. A variable's value disappears when the program ends, but a file's contents do not. Why — what is each one stored in?

4.4 When Things Go Wrong: Handling Errors

Files might not exist. Users type letters where you expected numbers. The internet drops. A beginner's program crashes at the first surprise; a professional's program **expects** trouble and handles it calmly. That is what exceptions are for.

ANALOGY — THE TRAPEZE AND THE SAFETY NET

A trapeze artist performs the daring move (the `try`), but a **net is stretched below** in case of a fall (the `except`). The net does not stop them attempting the trick — it stops a slip from becoming a disaster. `try / except` is your safety net: attempt the risky thing, and have a plan ready if it fails.

WHERE THE WORDS COME FROM

Exception — Latin *excipere*, “to take out” (*ex-* out + *capere* to take). An exception is an event “taken out” of the normal flow — the unusual case that interrupts the ordinary path.

try / except — You *try* (Old French *trier*, to attempt) an action, and you make an *exception* for when it fails. **finally** is the Latin *finalis* — the ending that always happens, success or failure.

```
# Without a net: this CRASHES the whole program
age = int(input("Your age? ")) # type "ten" and it explodes

# With a net: catch the specific problem and recover
try:
    age = int(input("Your age? "))
    print(f"In ten years you'll be {age + 10}.")
except ValueError:
    print("That wasn't a whole number — try again with digits.")
finally:
    print("Thanks for using the age tool.") # runs either way
```

```
Your age? ten
That wasn't a whole number — try again with digits.
Thanks for using the age tool.
```

PRO TIP — CATCH THE SPECIFIC EXCEPTION, NOT EVERYTHING

Prefer `except ValueError:` over a bare `except:`. Catching one named error means you only handle the problem you anticipated and genuine bugs still surface loudly. Common names worth knowing: `ValueError` (wrong kind of value), `FileNotFoundError` (missing file), `KeyError` (missing dictionary key), `ZeroDivisionError` (divide by zero).

WATCH OUT — DON'T SILENCE ERRORS YOU DIDN'T EXPECT

A bare `except: pass` swallows every error silently, including the typo three lines down. Your program limps on in a broken state and you have no idea why. Catch only what you mean to handle; let the unexpected speak up.

✓ KNOWLEDGE CHECK — ERRORS

1. What goes in the `try` block versus the `except` block?
2. When does the `finally` block run — on success, on failure, or both?
3. Why is `except ValueError:` usually better than a bare `except:` ?

4.5 Structured Memory: JSON

Plain text files are fine for prose, but how do you save a *dictionary* — a whole structure of names, ages, and lists — and load it back exactly as it was? The answer almost everyone uses is **JSON**.

ANALOGY — THE FLAT-PACK FURNITURE BOX

You cannot post a built bookshelf through a letterbox, so it ships flat-packed: disassembled into labelled parts with instructions, then rebuilt at the other end. JSON does this for data. `json.dumps` flat-packs your dictionary into a tidy text string for storage or sending; `json.loads` reads the instructions and rebuilds the exact same dictionary. Same furniture, just collapsed for travel.

WHERE THE WORD COMES FROM

JSON — *JavaScript Object Notation*. It was born from the way the JavaScript language writes objects, but proved so clean and readable that nearly every language and web service adopted it. *Notation* is from Latin *notare*, “to mark down” — a way of marking down structured data as text. The matching method names are **dump-string** (`dumps`) and **load-string** (`loads`).

```
import json

student = {
    "name": "Sana",
    "age": 16,
    "subjects": ["Maths", "Physics", "CS"],
    "active": True
}

# SAVE the dictionary to a file as JSON
with open("student.json", "w") as f:
    json.dump(student, f, indent=2)  # indent=2 keeps it human-readable

# LOAD it back — you get a real dictionary again
with open("student.json", "r") as f:
    loaded = json.load(f)

print(loaded["name"], "studies", loaded["subjects"])
```

```
Sana studies ['Maths', 'Physics', 'CS']
```

DUMP VS DUMPS — THE TINY S

`json.dump(data, file)` writes straight **to a file**. `json.dumps(data)` returns a **string** (the trailing **s** is for “string”) — handy when sending data over the internet rather than saving it. The same pair exists for reading: `json.load` from a file, `json.loads` from a string. You will see JSON again the moment you call an API — it is the language services speak.

✓ KNOWLEDGE CHECK — JSON

1. What does JSON let you do that a plain `.txt` file makes awkward?
2. What is the difference between `json.dump` and `json.dumps` ?
3. After `json.load` , what Python type do you typically get back, and how do you reach a value inside it?

4.6 Talking to the World: What Is an API?

Your program is about to do something that would have looked like magic a few years ago: ask a powerful AI a question and get an answer back. To do that, you need one more idea — the **API** — the way one program talks to another across the internet.

ANALOGY — ORDERING AT A RESTAURANT

You don't walk into the kitchen and cook your own meal. You read a **menu** (the list of what you can ask for), tell the **waiter** your order in a precise, agreed format, and the kitchen sends back a finished dish. You never see how the kitchen works inside — you just need to know how to order. An **API** (Application Programming Interface) is exactly that waiter: a fixed menu of requests one program agrees to answer, so other programs can ask without knowing the kitchen's secrets.

WHERE THE WORDS COME FROM

API — *Application Programming Interface*. An **interface** (Latin *inter*- “between” + *facies* “face”) is the shared face where two things meet and agree how to interact — like the dashboard between you and a car. The API is the agreed meeting-face between your program and someone else's service.

When your program calls an API it sends a **request** over the internet and receives a **response** — usually as JSON, which is exactly why you learned it on the previous page. You ask in a structured way; you get structured data back.

■ What is an LLM?

The service we will call is a **Large Language Model** (LLM) — the kind of AI behind modern chatbots. In plain terms: it was trained on an enormous amount of text and learned to predict what words should come next, so well that it can answer questions, explain ideas, write, and reason. You do not run this giant model on your laptop; it lives on powerful servers, and the API is how you send it a prompt and receive its reply.

THE MENTAL MODEL THAT MATTERS

You send the model a list of **messages** — each with a *role* (**system** to set its behaviour, **user** for what the person says) and some text. The model returns a message back. That request-and-reply, repeated, is the whole foundation of every chatbot you have ever used. Everything else is decoration on top of that loop.

✓ **KNOWLEDGE CHECK — APIS**

1. Using the restaurant analogy, what plays the role of the menu, the waiter, and the dish?
2. What format does data usually travel in when you call a web API — and where did you already meet it?
3. In one sentence, what is a Large Language Model doing when it answers you?

4.7 Your First AI Call — Safely

Now we connect it all. Three steps: get a key, store it safely, and make the call. The middle step is the one most beginners get wrong — and getting it wrong can cost real money — so we treat it with the seriousness it deserves.

■ Step 1 — Get an API key

Create an account at the OpenAI platform site and generate an **API key** from the dashboard. It looks like a long string beginning with `sk-`. Using the API requires credit on the account; a small top-up goes a very long way for the tiny experiments in this book.

ANALOGY — YOUR KEY IS YOUR HOUSE KEY

An API key identifies *you* and bills *you*. Posting it in your code is like taping your house key to your front door with your address on it. Anyone who finds it can walk in and spend your money. Treat the key like a password, because that is exactly what it is.

WATCH OUT — THE SINGLE MOST IMPORTANT RULE IN THIS CHAPTER

Never write your API key directly in your code. Not even “just for testing”. The instant that code is shared, screenshotted, or pushed to GitHub, bots find the key within *minutes* and run up charges. Keys belong in the **environment**, never in the source file. There is no exception to this rule.

■ Step 2 — Store the key in your environment

An **environment variable** is a value your operating system holds outside your program. Your code asks for it by name; the secret itself never appears in the file you write or share.

WHERE THE WORD COMES FROM

Environment — Old French *environ*, “around, surrounding”. Environment variables are the settings that *surround* a running program — the conditions of the room it runs in, separate from the program's own text. Your secret lives in the room, not in the script.

```
# Set the variable in your TERMINAL (it stays out of your code)
# macOS / Linux:
export OPENAI_API_KEY="sk-your-key-here"
# Windows (PowerShell):
$env:OPENAI_API_KEY="sk-your-key-here"
```

PRO TIP — THE .ENV FILE + GITIGNORE HABIT

For real projects, keep secrets in a file called `.env` and load them with the `python-dotenv` package (`pip install python-dotenv`). Then add a line saying `.env` to a file named `.gitignore` so the secret file is **never** uploaded when you share code. Secrets in `.env` , `.env` in `.gitignore` — commit that pair of habits to memory.

■ Step 3 — Make the call

The OpenAI library automatically looks for `OPENAI_API_KEY` in your environment, so your code never mentions the secret at all:

```
from openai import OpenAI

client = OpenAI()  # reads OPENAI_API_KEY from the environment automatically

response = client.chat.completions.create(
    model="gpt-4o-mini",  # a small, fast, inexpensive model
    messages=[
        {"role": "system", "content": "You are a friendly tutor for teenagers."},
        {"role": "user", "content": "Explain what a variable is in one sentence."}
    ]
)

print(response.choices[0].message.content)
```

A variable is a labelled box where your program stores a value so it can use or change it later by name.

READING THE RESPONSE

The reply is an object. The text you want is nested at `response.choices[0].message.content` — the `[0]` because the API can return several alternative answers and you are taking the first. Wrap real calls in `try / except` (Section 4.4): networks fail, and you do not want one hiccup to crash your whole app.

✓ KNOWLEDGE CHECK — CALLING OPENAI

1. Why must an API key *never* be written directly into your source code? What can go wrong?
2. Where should the key live instead, and how does `OpenAI()` find it without you typing it?
3. What is the purpose of the `system` message compared with the `user` message?
4. Where in the response object is the actual text of the reply found?

4.8 From One Call to a Conversation

A single question is a start, but a real assistant *remembers what was said*. The leap from one-off call to genuine conversation is smaller than you think: you simply keep a list of messages and add to it each turn.

ANALOGY — THE RUNNING TRANSCRIPT

Imagine a meeting where someone writes down every line spoken. Before anyone replies, they re-read the whole transcript so their answer fits the conversation. The model has no memory between calls — so **you** keep the transcript (a list of messages) and hand the whole thing over each time. That growing list is the conversation's memory.

Here is a complete command-line chatbot that remembers context. Type it, run it, and talk to it — then read how it works.

```
from openai import OpenAI

client = OpenAI()

# The transcript starts with the system instruction
messages = [
    {"role": "system", "content": "You are a patient study buddy. Keep answers short."}
]

print("Study buddy ready! Type 'quit' to leave.\n")

while True:
    user_text = input("You: ")
    if user_text.lower() == "quit":
        print("Goodbye — keep studying!")
        break

    messages.append({"role": "user", "content": user_text})

    try:
        reply = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages
        )
        answer = reply.choices[0].message.content
    except Exception as e:
        print(f"Something went wrong: {e}")
        continue

    print(f"Buddy: {answer}\n")
    messages.append({"role": "assistant", "content": answer})  # remember its own reply
```

Study buddy ready! Type 'quit' to leave.

You: What is a loop?

Buddy: A loop repeats a block of code so you don't write it many times.

You: Give me a tiny example of the first kind.

Buddy: `for i in range(3): print(i)` – it prints 0, 1, 2.

WHY IT REMEMBERS

Notice we append *both* the user's line **and** the assistant's reply back into `messages`. That is why the second question (“the first kind”) makes sense — the whole transcript travels with every call. Remove those appends and the bot forgets everything between turns.

PRO TIP — YOU NOW HAVE EVERY INGREDIENT

Look at what this 30-line program uses: a `while` loop, `if / break`, a list, dictionaries, `.append()`, string methods, `try / except`, and an API call. Every single piece came from Weeks 1–4. Nothing here is new — you are just combining tools you already own.

4.9 Capstone Project — The AI Study Coach

This is the summit. You will build a real application that generates flashcards on any topic using AI, quizzes you on them, tracks your score, and saves your progress to a file so it survives between sessions. It uses **everything**: classes, the four pillars, files, JSON, error handling, and the OpenAI API. This is no longer a toy — it is a tool you would actually use.

THE DESIGN — THINK IN NOUNS FIRST

Before writing code, name the nouns (Week 3 thinking). We have a **Flashcard** (a question and its answer) and a **StudyCoach** (the brain that generates cards, runs the quiz, and saves progress). Two clean classes, each with clear data and behaviour. Notice how the AI is just *one method* inside the coach — encapsulated, like an engine under a car's bonnet.

■ The complete program

```

import json
from openai import OpenAI

class Flashcard:
    """One question and its answer – a single noun in our problem."""
    def __init__(self, question, answer):
        self.question = question
        self.answer = answer

    def __str__(self):
        return f"Q: {self.question}"

    def to_dict(self):
        return {"question": self.question, "answer": self.answer}

class StudyCoach:
    """Generates cards with AI, quizzes the user, and saves progress."""
    def __init__(self, topic):
        self.topic = topic
        self.cards = []
        self.score = 0
        self.client = OpenAI()

    def generate_cards(self, how_many=5):
        """Ask the AI for flashcards and turn the reply into objects."""
        prompt = (
            f"Create {how_many} flashcards about '{self.topic}'. "
            "Reply ONLY with a JSON list of objects, each with "
            "'question' and 'answer' keys. No extra text."
        )
        try:
            reply = self.client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "user", "content": prompt}]
            )
            raw = reply.choices[0].message.content
            data = json.loads(raw)  # text -> list of dicts
            self.cards = [Flashcard(c["question"], c["answer"]) for c in data]
            print(f"Made {len(self.cards)} cards on {self.topic}!\n")
        except json.JSONDecodeError:
            print("The AI's reply wasn't valid JSON – try running again.")
        except Exception as e:
            print(f"Could not reach the AI: {e}")

    def quiz(self):
        """Loop through the cards and score the user."""
        for i, card in enumerate(self.cards, start=1):
            print(f"Card {i}: {card.question}")
            input(" (press Enter to see the answer) ")
            print(f" Answer: {card.answer}")
            got_it = input(" Did you get it? (y/n) ").lower()
            if got_it == "y":
                self.score += 1

```

```

        print()
        print(f"Final score: {self.score} / {len(self.cards)}")

    def save(self, filename="progress.json"):
        """Write topic, score, and cards to a file so progress survives."""
        record = {
            "topic": self.topic,
            "score": self.score,
            "cards": [card.to_dict() for card in self.cards]
        }
        with open(filename, "w") as f:
            json.dump(record, f, indent=2)
        print(f"Progress saved to {filename}.")

# ===== Run the coach =====
if __name__ == "__main__":
    topic = input("What shall we study today? ")
    coach = StudyCoach(topic)
    coach.generate_cards(5)
    coach.quiz()
    coach.save()

```

```

What shall we study today? the water cycle
Made 5 cards on the water cycle!

```

```

Card 1: What is evaporation?
    (press Enter to see the answer)
    Answer: When liquid water turns into vapour and rises into the air.
    Did you get it? (y/n) y

```

```

... (four more cards) ...
Final score: 4 / 5
Progress saved to progress.json.

```

WHY THIS DESIGN IS GOOD — THE PILLARS IN ACTION

Encapsulation: the `StudyCoach` hides the messy AI call behind a clean `generate_cards()` method — callers don't see the prompt or the JSON parsing. **Abstraction:** from the outside the coach is just “make cards, quiz me, save”. **Composition:** a coach *has* many `Flashcard` objects. Each class has one job. This is exactly how professional code is structured.

WATCH OUT — AI REPLIES AREN'T ALWAYS PERFECT JSON

Sometimes a model wraps its JSON in chatter or code-fences despite instructions. That is precisely why `generate_cards` catches `json.JSONDecodeError` — so a malformed reply gives a friendly message instead of a crash. Defensive code around anything coming from outside your program is the mark of a real developer.

 CAPSTONE EXTENSIONS — MAKE IT YOURS

1. **WARM-UP** Add a `load(filename)` method to `StudyCoach` that reads a saved `progress.json` back into cards, so a student can resume.
2. **CORE** Add a `difficulty` parameter to `generate_cards` (“easy”, “hard”) and weave it into the prompt sent to the AI.
3. **CORE** Track *which* cards were missed and offer a “review only the ones you got wrong” round at the end.
4. **STRETCH** Add an `explain(card)` method that, when the student gets one wrong, asks the AI for a simpler explanation of that specific answer.
5. **STRETCH** Create a `Subject` class that holds several `StudyCoach` sessions across topics and reports an overall progress summary — a small class hierarchy of your own design.

4.10 Practise & Where to Go Next

EXERCISES — REACHING OUT

1. **WARM-UP** **Module explorer.** Import `random` and write a program that prints a random motivational study quote from a list of five you wrote yourself.
2. **WARM-UP** **Note saver.** Write a program that asks the user for a note and appends it (with today's date from `datetime`) to a file called `notes.txt`.
3. **CORE** **Safe calculator.** Ask for two numbers and divide them, using `try / except` to handle both non-numbers and division by zero with friendly messages.
4. **CORE** **Contacts in JSON.** Build a tiny program that lets the user add name/phone pairs to a dictionary and saves it to `contacts.json`, reloading it on the next run so contacts persist.
5. **CORE** **AI fact giver.** Using the OpenAI API, write a program that asks the user for a topic and prints one surprising fact about it. Wrap the call in `try / except`.
6. **STRETCH** **AI summariser.** Read a paragraph of text from a `.txt` file and ask the AI to summarise it in three bullet points.
7. **STRETCH** **Themed chatbot.** Take the chatbot from 4.8 and give it a personality entirely through its `system` message (a pirate tutor, a Shakespearean coach). Change only that one line.
8. **STRETCH** **Quiz master class.** Design a `Quiz` class that loads questions from a JSON file, asks them, scores the user, and saves the high score — no AI required, pure OOP + files.

Where to go next — your path beyond this book

You began four weeks ago unable to print a line. You can now design classes, handle errors, persist data, and command an AI. That is a real foundation. Here is where the road leads:

If you enjoyed...	Explore next
Building the AI app	Prompt design, the <code>requests</code> library, and other APIs (weather, maps, music)
Making things visual	Flask or FastAPI to put your code behind a web page
Working with data	pandas and matplotlib for analysing and charting real datasets
Games and graphics	pygame to build playable games with your OOP skills
Doing it properly	Writing tests with <code>pytest</code> , and using Git for version control

PRO TIP — THE ONLY WAY TO KEEP IT

Reading code is not the same as writing it. Pick **one** small project you actually care about — a tracker for your hobby, a quiz for your friends, a tool for a class — and build it badly. Then improve it. A finished imperfect project teaches more than a hundred perfect tutorials. The students who keep coding are simply the ones who kept building.

CONGRATULATIONS — YOU FINISHED THE PLAYBOOK

Four weeks ago, “variable”, “class”, and “API” were just words. Now they are tools in your hands, and you know where each one came from and why it works the way it does. You did not memorise Python — you *understood* it. Keep the answer keys and cheat sheet that follow close by, keep building things that interest you, and welcome to a skill that will serve you for the rest of your life. The terminal is waiting. Go make something.



APPENDIX A

Knowledge Check Answers

Check yourself honestly – but only after you have tried to answer from memory. An answer you struggled toward sticks; an answer you merely read evaporates.

Appendix A — Knowledge Check Answers

■ Setup

1. Adding Python “to PATH” lets you type `python` in any terminal and have the computer find it, instead of needing its full folder location every time. 2. The REPL runs code line by line and shows results instantly — ideal for quick experiments; a `.py` script is a saved file of many lines you run all at once — ideal for real programs you keep. 3. Read, Eval(uate), Print, Loop.

■ Variables

1. Take the current value of `total`, add 10 to it, and store the result back into `total` — the `=` is assignment, not equality. 2. Python is case-sensitive, so capital and lowercase letters are treated as entirely different names. 3. `2nd_place` is invalid (cannot start with a digit) and `my-score` is invalid (the hyphen reads as minus); `second_place` and `myScore` are both valid.

■ Types & Operators

1. `"7" + "7"` gives `"77"` — `+` on strings glues them; `7 + 7` gives `14` because `+` on numbers adds. 2. `17 % 5` is `2` (the remainder after dividing); modulo captures “what's left over” — the same idea as hours wrapping on a clock. 3. `False` — `and` needs both sides true, and `3 == 4` is false. 4. `bool`, named after George Boole, the mathematician who formalised true/false logic.

■ Strings

1. `s[0]` is `"S"`, `s[-1]` is `"r"`, `s[1:4]` is `"umm"`. 2. A slice includes the start index but stops *before* the end index, so `1:4` covers positions 1, 2, 3 — three characters. 3. Immutable means a string can never be changed in place; methods like `.replace()` return a brand-new string, so you must capture it: `s = s.replace(...)`.

■ Decisions

1. Because `input()` always returns text, and comparing text to a number either errors or behaves wrongly — `int()` converts it to a real number first. 2. At most one branch runs; Python takes the first true condition and skips the rest. 3. Indentation — the spaces at the start of a line are what tell Python a line belongs inside the block.

■ Loops

1. Three numbers: `0, 1, 2` — it starts at 0 and stops before 3. 2. A `for` loop, because you are walking through a collection of known items; `while` is for repeating until some condition changes. 3. A loop whose condition never becomes false, so it runs forever; the preventing question is “what inside this loop will eventually make the condition stop being true?”

■ Collections

1. A tuple — its immutability guarantees the waypoints cannot be altered by accident. 2. A set, because it automatically discards duplicates, leaving only distinct items. 3. A key is the label you look something up by; the value is what you get back — e.g. key `"name"` → value `"Sana"`. 4. It lets you loop over key and value together: `for name, age in ages.items():`.

■ Functions, Scope & Comprehensions

1. A parameter is the name in the function definition (the empty slot); an argument is the actual value you pass in when calling. 2. `print()` only displays text on screen; without `return` the function hands back `None`, so `x` receives nothing useful. 3. Scope is the region where a name is visible; a local variable exists only while its function runs and is invisible outside, keeping functions self-contained. 4. `[len(word) for word in ["hi", "world", "ok"]]` → `[2, 5, 2]`.

■ Classes, `__init__` & `self`

1. A class is a blueprint describing structure and behaviour; an object is one actual thing built from that blueprint. 2. `__init__` runs automatically the moment an object is created; its job is to set up the object's starting attributes. 3. Through `self` — each call carries the specific object, so `self.name` is that dog's own name. 4. An attribute is data the object holds (a noun); a method is an action it can perform (a verb, a function inside the class).

■ The Four Pillars

1. Encapsulation (a capsule/car you drive without touching the engine), Inheritance (a family tree passing traits down), Polymorphism (one word, many shapes — `.area()` on any shape), Abstraction (a TV remote: simple buttons hiding complexity). 2. It reaches in and sets the data directly, bypassing the rules the class should enforce (a balance should never go to nonsense) — the object can no longer protect its own state. 3. It runs the parent class's setup so the child inherits all the parent's initialisation without repeating it. 4. Polymorphism — literally “many forms” (Greek *poly* + *morphē*).

■ Dunder & Beyond

1. Defining `__str__` lets you control the friendly text shown when the object is printed, instead of a cryptic memory address. 2. A class variable is shared by every instance (e.g. `school = "Green Valley"` for all students); an instance variable is unique to each object (e.g. each student's own `name`). 3. `@property` lets you run code (validation, calculation) when an attribute is read, while still accessing it with simple dot syntax rather than a method call.

■ Modules

1. `import math` brings in the whole module, so you call `math.sqrt(...)`; `from math import sqrt` brings in just that one name, so you call `sqrt(...)` directly. 2. A module is a single `.py` file of code; a package is a folder bundling several related modules together. 3. The `as` gives the module a shorter nickname, so you type `dt.date` instead of `datetime.date` every time.

■ pip & venv

1. In your terminal, not the Python prompt. 2. It gives each project its own isolated set of packages so their versions can't clash. 3. It lists every package and exact version the project uses, so anyone can recreate the identical setup with `pip install -r requirements.txt`.

■ Files

1. `with` automatically closes the file when the block ends — even if an error occurs — so you can never forget to close it. 2. `"w"` erases the file and writes fresh; `"a"` keeps the existing contents and adds to the end. 3. A variable lives in memory (RAM), which is cleared when the program ends; a file lives on disk, which persists after the program and after a reboot.

■ Errors

1. The risky action goes in `try`; the recovery plan for when it fails goes in `except`. 2. Both — `finally` always runs whether the `try` succeeded or failed. 3. Catching a specific error handles only the problem you anticipated and lets genuine, unexpected bugs surface loudly instead of being silently swallowed.

■ JSON

1. It lets you save and reload whole structures — dictionaries, lists, nested data — intact, which plain text makes clumsy. 2. `json.dump` writes directly to a file; `json.dumps` returns a string (the `s` is for “string”). 3. Usually a dictionary; you reach a value with its key, e.g. `data["name"]`.

■ APIs

1. The menu is the list of requests the API accepts; the waiter is the API carrying your request and response; the dish is the data returned. 2. JSON — which you met in Section 4.5. 3. It is predicting the most fitting next words based on patterns learned from huge amounts of text, well enough to answer, explain, and reason.

■ Calling OpenAI

1. Because anyone who sees the key can spend your money; shared or pushed code is scanned by bots within minutes. 2. In an environment variable named `OPENAI_API_KEY`; `OpenAI()` reads it from the environment automatically, so the secret never appears in your code. 3. The `system` message

sets the assistant's overall behaviour and personality; the `user` message is the specific thing the person is asking right now. 4. At `response.choices[0].message.content`.

B

APPENDIX B

Exercise Solutions & Hints

There is rarely one “right” solution — if yours produces the correct result, it counts. Use these to compare approaches and unstick yourself, not to skip the struggle that does the teaching.

Appendix B — Exercise Solutions & Hints

■ Week 1 — Foundations

```
# 1. About me
name, age, subject = "Ada", 16, "Computer Science"
print(f"{name} is {age} and loves {subject}.")

# 2. Temperature converter
c = 25
f = c * 9/5 + 32
print(f"{c}C = {f}F")

# 4. Even or odd
n = int(input("A whole number: "))
print("even" if n % 2 == 0 else "odd")

# 5. Initials
full = "ada lovelace"
parts = full.split()
print(parts[0][0].upper() + "." + parts[1][0].upper() + ".")
```

Hints — Ex 3 (Rectangle): convert both inputs with `int()`; area is `w*h`, perimeter is `2*(w+h)`. Ex 6 (Grade): an `if/elif/else` chain checking `>= 90`, then `>= 80`, and so on. Ex 7: the error is a `ValueError` — Python can't turn the word into a number; that is exactly why conversion matters.

■ Week 2 — Building Blocks

```
# FizzBuzz – the classic
for n in range(1, 21):
    if n % 15 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)

# Word counter using a dictionary
text = "the cat sat on the mat the end"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)  # {'the': 3, 'cat': 1, ...}
```

Hints — Check `% 15` first in FizzBuzz, or the “FizzBuzz” case never fires. For a sum-of-a-list exercise, start `total = 0` before the loop and add inside it. For “unique items”, wrap a list in `set(...)`.

■ Week 3 — Objects

```
# A simple class with behaviour
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
    def __str__(self):
        return f"Rectangle {self.width}x{self.height}"

r = Rectangle(3, 4)
print(r, "has area", r.area())  # Rectangle 3x4 has area 12
```

Hints — For the OOP to-do rebuild, make a `TaskList` class holding a list of tasks, with `add`, `remove`, and `show` methods — the data and the functions that touch it now live together. For an inheritance exercise, define the shared parts in a base class and only the differences in each child, calling `super().__init__()` first.

■ Week 4 — Reaching Out

```
# Safe calculator
try:
    a = float(input("First number: "))
    b = float(input("Second number: "))
    print(f"Result: {a / b}")
except ValueError:
    print("Please type actual numbers.")
except ZeroDivisionError:
    print("You can't divide by zero!")

# AI fact giver
from openai import OpenAI
client = OpenAI()
topic = input("Topic: ")
try:
    r = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user",
                    "content": f"Tell me one surprising fact about {topic}."}]
    )
    print(r.choices[0].message.content)
except Exception as e:
    print(f"Couldn't reach the AI: {e}")
```

Hints — For the JSON contacts app, load with `json.load` inside a `try` that catches `FileNotFoundError` (start empty on first run). For the themed chatbot, change only the `system` message. For the `Quiz` class, store questions as a list of dicts and loop with `enumerate` exactly as the capstone does.



APPENDIX C

Quick-Reference Cheat Sheet

Everything in this book, compressed to fit beside your keyboard. Print this page and keep it where you code.

Appendix C — Cheat Sheet

■ Variables & types

```
name = "Sana"      # str (text)
age = 16           # int (whole number)
height = 1.6       # float (decimal)
active = True      # bool (True / False)
int("5"), str(5), float("5.0") # convert
```

■ Strings

```
s = "Summer"
s[0]      # 'S'    first
s[-1]     # 'r'    last
s[1:4]    # 'umm'  slice (stops before 4)
len(s)    # 6
s.upper(), s.lower(), s.strip(), s.split(), s.replace("a","b")
print(f"{name} is {age}") # f-string
```

■ Decisions & loops

```
if x > 10:
    ...
elif x == 10:
    ...
else:
    ...

for i in range(5):    # 0,1,2,3,4
    ...
while condition:
    ...               # use break to exit
```

■ Collections

```
nums = [1, 2, 3]      # list  - ordered, changeable
point = (4, 5)        # tuple - ordered, fixed
ages = {"Ana": 16}    # dict  - key:value pairs
seen = {1, 2, 3}      # set   - unique items
nums.append(4); ages["Bo"] = 17
for k, v in ages.items(): ...
```

■ Functions

```
def greet(name, excited=False):
    msg = f"Hi {name}"
    return msg + "!" if excited else msg

squares = [n*n for n in range(5)]    # comprehension
```

■ Classes (OOP)

```
class Dog(Animal):                # inherits from Animal
    species = "Canis"             # class variable (shared)
    def __init__(self, name):
        super().__init__()        # run parent setup
        self.name = name          # instance variable
    def bark(self):                # method
        return f"{self.name} says woof"
    def __str__(self):
        return f"Dog({self.name})"

d = Dog("Rex")
```

■ Files, JSON & errors

```
with open("f.txt", "w") as f:    # "r" read, "a" append
    f.write("hi")

import json
json.dump(data, f)               # save to file
data = json.load(f)              # load from file

try:
    risky()
except ValueError:
    handle()
finally:
    always_runs()
```

■ OpenAI API

```
from openai import OpenAI
client = OpenAI()                # key from OPENAI_API_KEY env var
r = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are helpful."},
        {"role": "user", "content": "Hello!"}
    ])
print(r.choices[0].message.content)
```



APPENDIX D

Glossary of Word Origins

*Every technical term was once an ordinary word borrowed for a new job.
Knowing where a word came from is often the fastest way to remember
what it does.*

Appendix D — Glossary of Word Origins

Term	Origin & meaning
Python	Named not after the snake but after the British comedy troupe <i>Monty Python's Flying Circus</i> — creator Guido van Rossum wanted a name that was short, unique, and a little mysterious.
Program	Greek <i>pro-</i> (before) + <i>graphein</i> (to write) — literally “written beforehand”, a set of instructions laid down in advance.
Variable	Latin <i>variare</i> , “to change” — a named container whose stored value can vary.
Boolean	After George Boole (1815–1864), who built the algebra of true and false that all logic runs on.
Modulo	Latin <i>modulus</i> , “a small measure” — the remainder left after measuring one number into another.
String	A “string” of characters threaded together in order, like beads on a line.
Index	Latin for “pointer/forefinger” — the number that points to a position in a sequence.
Bug / Debug	From a real moth found jamming an early computer in 1947; removing it was “debugging”.
Iterate	Latin <i>iterare</i> , “to repeat” — to go through items one after another.
Tuple	From the <i>-ple</i> in quintuple, sextuple — a fixed, ordered group of values.
Set	Borrowed from mathematics (Georg Cantor's set theory) — a collection of distinct elements with no duplicates.
Function	Latin <i>fungi</i> , “to perform” — a named block that performs a job and hands back a result.
Parameter	Greek <i>para-</i> (beside) + <i>metron</i> (measure) — a value measured out and passed alongside a call.
Scope	Greek <i>skopos</i> , “target/range of view” — the region of code where a name is visible.
Class	Latin <i>classis</i> , a division or group — a category whose members share the same structure.
Object / Instance	<i>Object</i> : Latin <i>obicere</i> , “thrown before” the mind — a thing. <i>Instance</i> : Latin <i>instantia</i> , a particular case of the class.
Encapsulation	Latin <i>capsula</i> , “little box” — sealing data and its methods together in one capsule.
Inheritance	Latin <i>hereditare</i> , “to inherit” — a child class receiving traits from a parent.
Polymorphism	Greek <i>poly</i> (many) + <i>morphē</i> (form) — one name taking many forms.

Abstraction	Latin <i>abstrahere</i> , “to draw away” — hiding detail to show only the essentials.
Module	Latin <i>modulus</i> , “small measure” — a standard, self-contained piece that slots together with others.
Import	Latin <i>importare</i> , “to carry in” — carrying code in from another file.
Library	Latin <i>librarium</i> , “a chest for books” — a collection you borrow code from.
pip	Recursive joke acronym “Pip Installs Packages”; also a small seed.
File	Latin <i>filum</i> , “a thread” — documents once threaded on a wire to keep order.
Exception	Latin <i>excipere</i> , “to take out” — an event taken out of normal flow.
JSON	<i>JavaScript Object Notation</i> — a clean text way to mark down structured data, now used everywhere.
API	<i>Application Programming Interface</i> — an <i>interface</i> (inter + facies, “between-face”) where two programs agree to meet.
Environment	Old French <i>environ</i> , “surrounding” — the settings that surround a running program.

ONE LAST THOUGHT

You now hold not just the *what* of Python but the *why* behind its words. That understanding is what separates someone who copies code from someone who truly writes it. Keep this book nearby, keep building, and keep asking where things come from — in programming and everywhere else.